INTEGRANTES:
Manuel Astudillo Ayala,
Benjamín Muñoz Balboa,
Carlos Moreno Rojas y
Carlos Rojas Valdés.

MINECRAFT MANAGEMENT SYSTEM™

Presentación de proyecto de implementación en laboratorio:
Gestión de datos basada en clases.

PROFESOR:
Bruno Faúndez Valenzuela

DOMINIO ABORDADO

La gestión de "Trading Halls" en mundos multijugador de Minecraft, donde la gran cantidad de aldeanos genera desorden es una complicación. Los jugadores pierden tiempo buscando intercambios específicos, olvidan las coordenadas exactas y sufren alteraciones o pérdidas no controladas provocadas por otros usuarios.

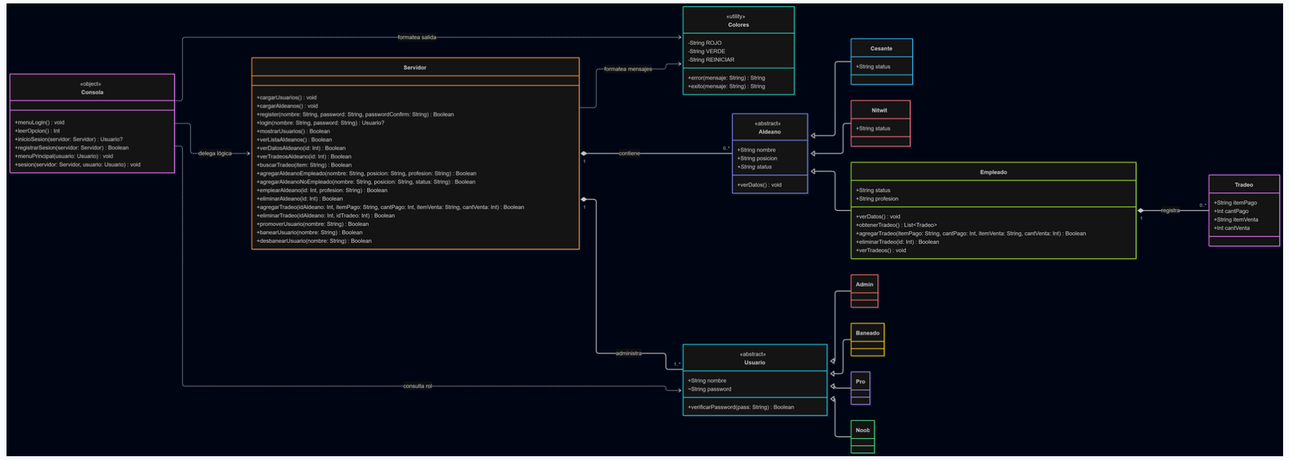
Hoy en día, los jugadores intentan organizar esta zona usando carteles de madera o simplemente confiando en su memoria. Esto genera varios problemas, entre ellos la perdida de tiempo, de ubicacion y falta de administracion.

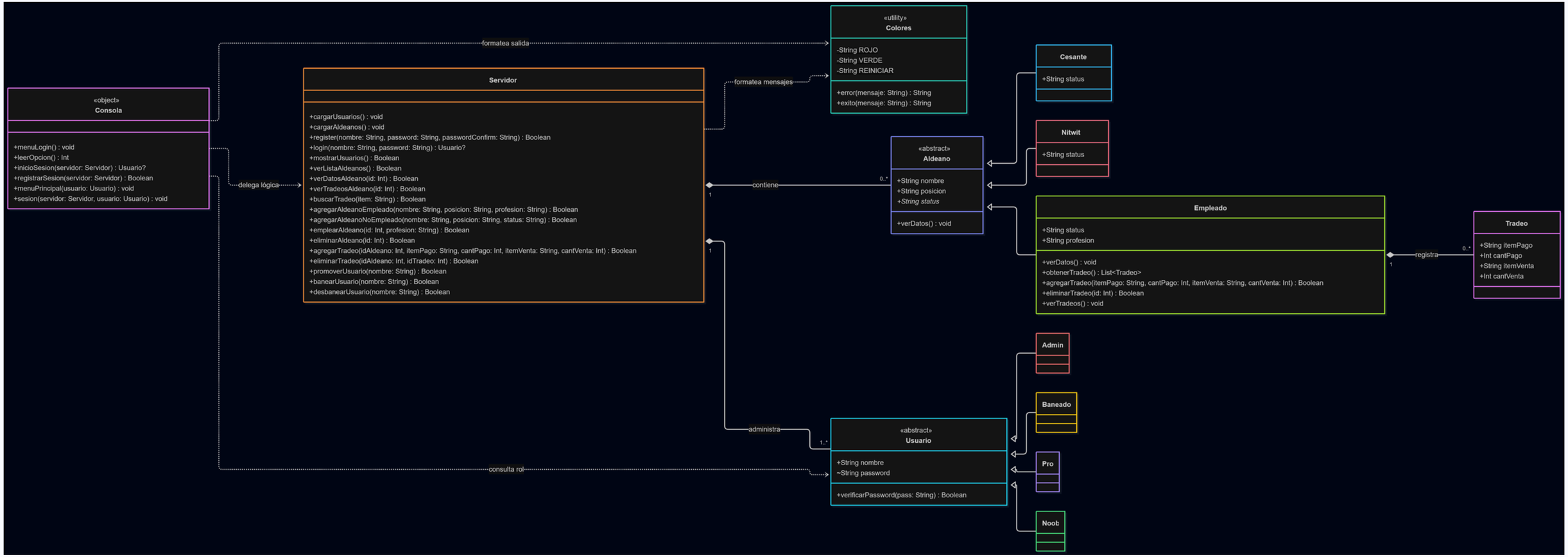


Trading Halls

SOLUCIÓN PROPUESTA Y MODELO DE CLASES

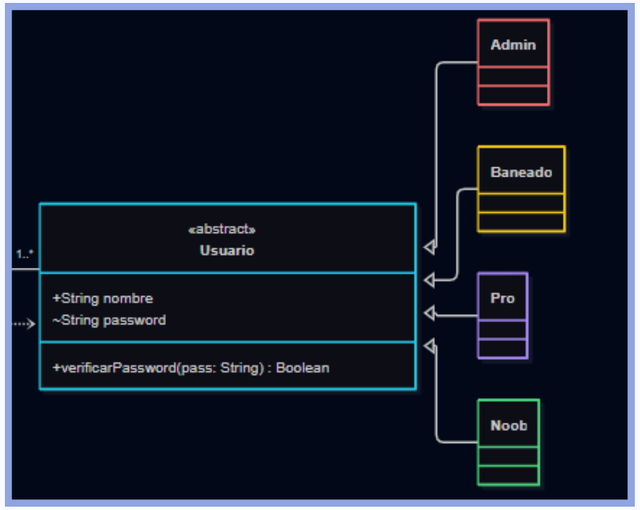
Se propone un sistema de registro por consola que estructura la información y establece control de acceso mediante rangos (Noob, Pro, Admin). El modelo de clases divide la aplicación en una interfaz de usuario interactiva (Consola), un motor central gestor (Servidor) y entidades de datos puras (Usuario, Aldeano, Tradeo).

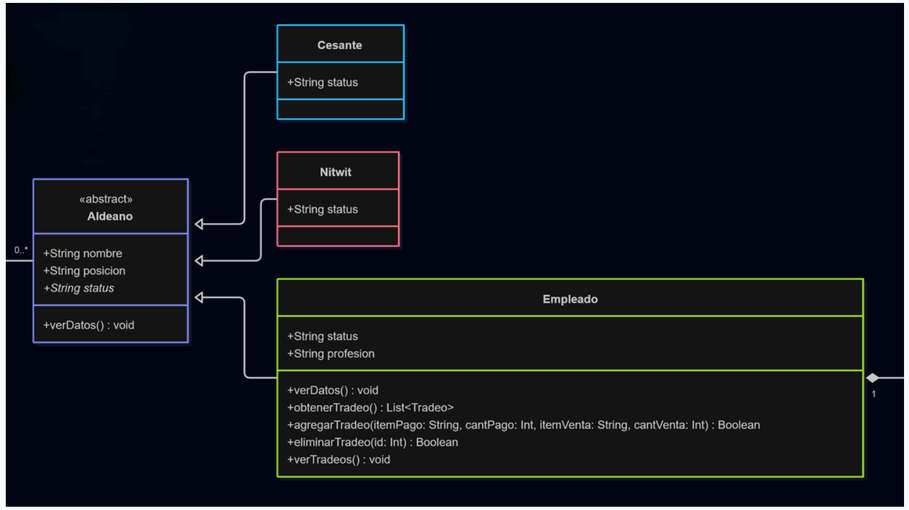


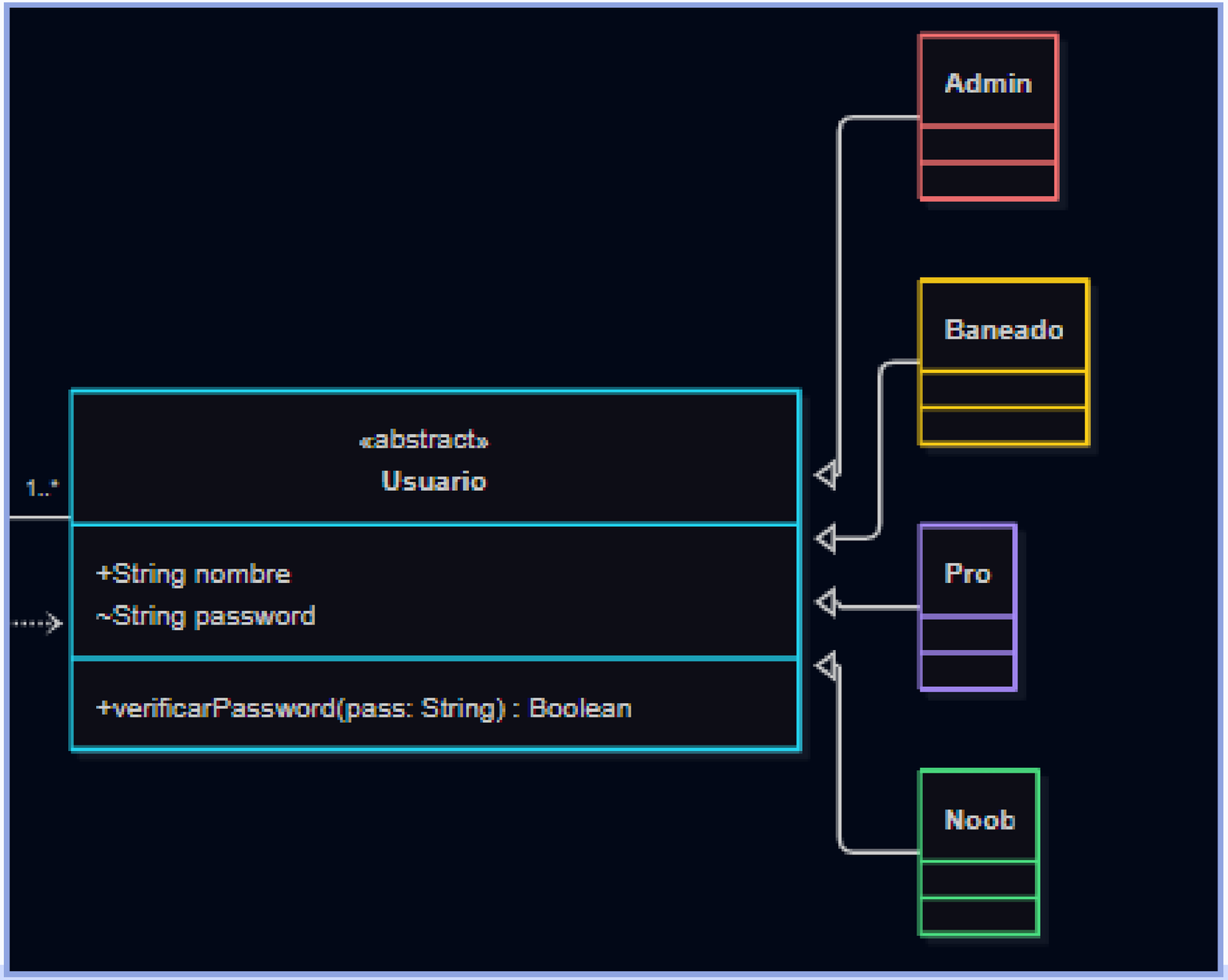

«object»
Consola
+menuLogin() : void
+leerOpcion() : Int
+inicioSesion(servidor: Servidor) : Usuario?
+registrarSesion(servidor: Servidor) : Boolean
+menuPrincipal(usuario: Usuario) : void
+sesion(servidor: Servidor, usuario: Usuario) : void
formatea salida
delega lógica
consulta rol
«utility»
Colores
-String ROJO
-String VERDE
-String REINICIAR
+error(mensaje: String) : String
+exito(mensaje: String) : String
formatea mensajes
Servidor
+cargarUsuarios() : void
+cargarAldeanos() : void
+register(nombre: String, password: String, passwordConfirm: String) : Boolean
+login(nombre: String, password: String) : Usuario?
+mostrarUsuarios() : Boolean
+verListaAldeanos() : Boolean
+verDatosAldeano(id: Int) : Boolean
+verTradeosAldeano(id: Int) : Boolean
+buscarTradeo(item: String) : Boolean
+agregarAldeanoEmpleado(nombre: String, posicion: String, profesion: String) : Boolean
+agregarAldeanoNoEmpleado(nombre: String, posicion: String, status: String) : Boolean
+emplearAldeano(id: Int, profesion: String) : Boolean
+eliminarAldeano(id: Int) : Boolean
+agregarTradeo(idAldeano: Int, itemPago: String, cantPago: Int, itemVenta: String, cantVenta: Int) : Boolean
+eliminarTradeo(idAldeano: Int, idTradeo: Int) : Boolean
+promoverUsuario(nombre: String) : Boolean
+banearUsuario(nombre: String) : Boolean
+desbanearUsuario(nombre: String) : Boolean
contiene
1
0..*
«abstract»
Aldeano
+String nombre
+String posicion
+String status
+verDatos() : void
Cesante
+String status
Nitwit
+String status
Empleado
+String status
+String profesion
+verDatos() : void
+obtenerTradeo() : List<Tradeo>
+agregarTradeo(itemPago: String, cantPago: Int, itemVenta: String, cantVenta: Int) : Boolean
+eliminarTradeo(id: Int) : Boolean
+verTradeos() : void
registra
1
0..*
Tradeo
+String itemPago
+Int cantPago
+String itemVenta
+Int cantVenta
administra
1
1..*
«abstract»
Usuario
+String nombre
~String password
+verificarPassword(pass: String) : Boolean
Admin
Baneado
Pro
Noob

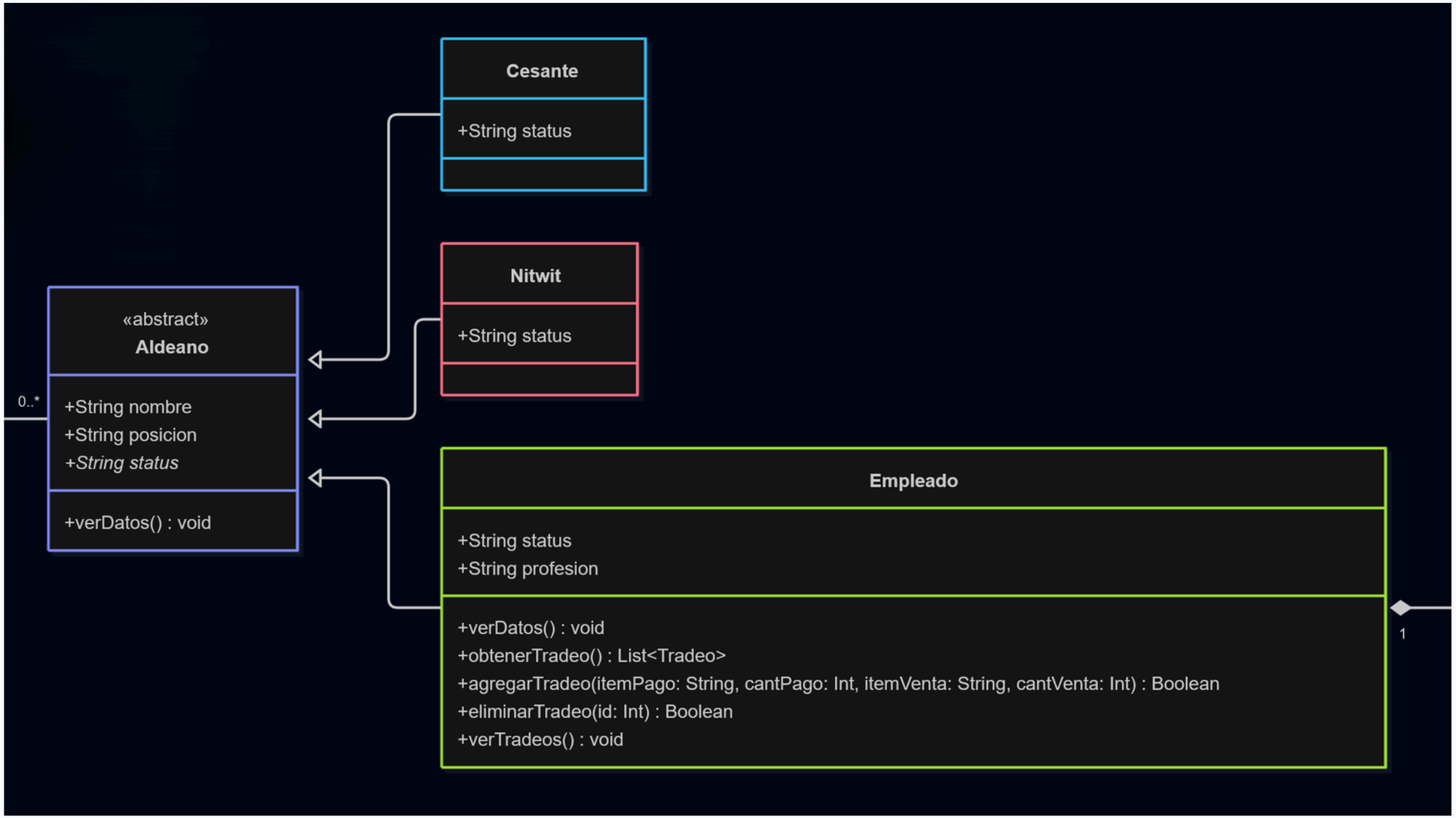
HERENCIA Y ABSTRACCIÓN

La abstracción se aplica en las clases Aldeano y Usuario para definir propiedades base (como nombre y posición) sin permitir su instanciación directa. La herencia se utiliza para crear variantes específicas: Empleado, Cesante y Nitwit extienden de Aldeano, mientras que los niveles de permiso Noob, Pro, Admin y Baneado extienden de Usuario.






Admin
Baneado
«abstract»
Usuario
1..*
+String nombre
~String password
+verificarPassword(pass: String) : Boolean
Pro
Noob


Cesante
+String status
Nitwit
+String status
«abstract»
Aldeano
+String nombre
+String posicion
+String status
+verDatos() : void
0..*
Empleado
+String status
+String profesion
+verDatos() : void
+obtenerTradeo() : List<Tradeo>
+agregarTradeo(itemPago: String, cantPago: Int, itemVenta: String, cantVenta: Int) : Boolean
+eliminarTradeo(id: Int) : Boolean
+verTradeos() : void
1

PATRÓN DE DISEÑO UTILIZADO

El proyecto emplea el patrón de diseño Singleton en la interfaz de usuario. Al declarar Consola mediante la palabra reservada object de Kotlin, el sistema garantiza la existencia de una única instancia global encargada de gestionar los menús del programa.

```kotlin
package helpers
import models.*

/**
 * Interfaz de consola para el "Villager Management System".
 *
 * Gestiona los menús, la captura de entrada estándar y la presentación de mensajes,
 * delegando la ejecución de la lógica de negocio y validación al [Servidor].
 */
object Consola {
    /**
     * Imprime el menú de inicio para usuarios no autenticados.
     */
    fun menuLogin() {
        println("\n=== VILLAGER MANAGEMENT SYSTEM ===")
```

REVISIÓN DEL CODIGO





INTEGRANTES:
Manuel Astudillo Ayala,
Benjamín Muñoz Balboa,
Carlos Moreno Rojas y
Carlos Rojas Valdés.

MUCHAS GRACIAS

PROFESOR:
Bruno Faúndez Valenzuela